

Dokumentasi Teknis Kode Sumber (Source Code) - Camelio Finance

Dokumen ini adalah rincian teknis tingkat lanjut (Deep Dive) yang membedah setiap file, fungsi, dan arsitektur kode sumber (Source Code) dari aplikasi Camelio Finance secara menyeluruh. Dokumen ini ditujukan untuk Software Engineer atau penilai teknis yang ingin meninjau arsitektur perangkat lunak dari aplikasi ini.

1. Arsitektur Pattern (MVVM & Service Locator)

Aplikasi Camelio Finance dirancang menggunakan kombinasi **Model-View-ViewModel (MVVM)** melalui package provider dan **Service Locator Pattern** menggunakan `get_it`.

- **Model:** Berada di folder `lib/data/models/`. Memetakan data dari Firestore JSON ke objek Dart.
 - **View:** Berada di folder `lib/ui/pages/` dan `lib/ui/widgets/`. Murni berisi kode Flutter (UI/UX) dan pasif menunggu data dari Provider.
 - **ViewModel:** Berada di folder `lib/providers/`. Bertugas sebagai jembatan yang menghubungkan View dan Data/Service.
 - **Service:** Berada di folder `lib/data/services/`. Menjalankan fungsi berat (API Call, eksekusi Database) yang kemudian dipanggil oleh Provider melalui `locator()`.
-

2. Bedah Lapisan Data (Data Layer)

Lapisan data bertanggung jawab untuk mengambil, menyimpan, dan memetakan data.

A. Data Models (`lib/data/models/`)

Setiap entitas utama di aplikasi memiliki representasi kelas Model agar *type-safe*.

1. **`user_model.dart`:** Blueprint profil user. Terdiri dari field dasar (`uid`, `name`, `email`, `monthlyBudget`, `balance`). Dilengkapi metode `fromJson` dan `toJson` untuk konversi data Firestore.
2. **`transaction_model.dart`:** Blueprint pencatatan kas. Menampung data nominal, kategori, catatan, serta List/Array detail barang belanjaan yang diekstrak oleh AI.
3. **`e_wallet_model.dart`:** Menampung data integrasi e-wallet, termasuk ikon dan status sinkronisasi.
4. **`expense_model.dart`:** Struktur data khusus untuk chart/grafik analitik keuangan.
5. **`notification_model.dart`:** Data log peringatan sistem dan peringatan AI Advisor.

B. Data Services (`lib/data/services/`)

Service didaftarkan ke `get_it` secara *lazy singleton* sehingga hanya diinisialisasi saat dibutuhkan.

1. **`gemini_ai_service.dart (Core AI)`:** Modul terbesar (10KB+) yang menangani logika *prompt engineering*. Terdapat fungsi `extractReceipt()` untuk mengirim URL Cloudinary ke Gemini Vision, dan `analyzeFinancials()` untuk AI Insight.
2. **`firestore_service.dart`:** Pembungkus (*wrapper*) untuk semua operasi CRUD (Create, Read, Update, Delete) ke Cloud Firestore. Memastikan data dikirim sesuai Model yang benar.
3. **`auth_service.dart`:** Logika Firebase Auth. Meliputi Login Email/Pass, Register, dan integrasi Google Sign-In (OAuth).

4. **gmail_service.dart**: Mengakses Gmail API via OAuth 2.0 untuk memindai email mutasi masuk dari bank atau dompet digital dan menyinkronkannya sebagai data transaksi.
 5. **cloudinary_service.dart**: Menggunakan REST API untuk mengunggah (POST) foto struk ke server Cloudinary sebelum foto diproses AI agar ukuran data jauh lebih kecil dan cepat.
 6. **csv_logger_service.dart**: Menyediakan fitur ekspor data transaksi ke format CSV untuk diunduh pengguna.
 7. **notification_service.dart**: Menginisialisasi *Flutter Local Notifications Plugin* untuk push-notification di HP pengguna.
-

3. Bedah Lapisan Logika Bisnis (Providers Layer)

Provider memegang *state* aplikasi. File-file ini berada di `lib/providers/` dan di-*inject* di `main.dart` menggunakan `MultiProvider`.

1. **auth_provider.dart**: Memegang status login (`isLoading`, `isAuthenticated`, `currentUser`). Menangani aliran layar dari Splash Screen ke Dashboard atau halaman Login.
 2. **e_wallet_provider.dart**: Modul raksasa (12KB) yang mengatur alur kompleks dari penambahan *wallet* eksternal hingga kalkulasi saldo kumulatif antar akun.
 3. **transaction_provider.dart**: Menarik data (`fetch`) riwayat transaksi, menghitung ulang total saldo (`income - expense`), dan menerapkan filter bulanan/kategori untuk grafik.
 4. **ai_insight_provider.dart**: Mengelola logika *loading state* saat AI Gemini sedang berpikir/merespons input teks pengguna (Chatbot).
 5. **settings_provider.dart**: Mengambil pengaturan lokal via `SharedPreferences` seperti preferensi Dark Mode.
 6. **notification_provider.dart**: Mengontrol pembacaan notifikasi (`Read/Unread`).
-

4. Bedah Lapisan Antarmuka (UI Layer)

Folder `lib/ui/` dipisah menjadi `pages` dan `widgets`. Semua desain mematuhi aturan estetika *Glassmorphism* yang diatur secara global di `core/theme/app_theme.dart`.

A. Pages / Layar Utama (`lib/ui/pages/`)

- **splash/**: Pintu gerbang aplikasi, memverifikasi sesi login secara asinkron.
- **auth/**: Terdiri dari `login_page.dart` dan `register_page.dart`.
- **main/**: Memuat `main_page.dart` yang memegang logika Bottom Navigation Bar. Layar ini adalah fondasi yang memuat tab Dashboard, Transaksi, dan Profil tanpa mematikan state layar lain.
- **dashboard/**: Pusat ringkasan kas (Card Saldo) dan grafik kecil (`fl_chart`).
- **transaction/**: Memiliki banyak rute, mulai dari Input Manual (`add_transaction_page.dart`), input suara, hingga OCR Camera (`scan_receipt_page.dart`).
- **ai_advisor/**: Halaman interaktif Chat AI mirip ChatGPT untuk konsultasi bisnis.
- **portfolio/ & budget/**: Halaman diagram mendetail dan manajemen limit dana per bulan.
- **e_wallet/**: Layar manajemen sinkronisasi dompet digital.
- **settings/**: Mengatur profil, cetak laporan (Report Generator PDF), dan izin perangkat (Microphone & Kamera).

B. Widgets Kustom (`lib/ui/widgets/`)

- `custom_text_field.dart`: Didesain khusus agar seragam di semua form (memiliki efek transisi fokus).
- `transaction_card.dart`: Komponen kartu list riwayat transaksi.

- **custom_popup.dart**: Menggantikan AlertDialog bawaan Flutter dengan desain yang lebih premium dan elegan.
-

5. Konfigurasi Sistem Utama (lib/main.dart & app.dart)

- **main.dart**: Titik masuk eksekusi (Entry point). Memanggil WidgetsFlutterBinding.ensureInitialized(), menginisialisasi Firebase App (DefaultFirebaseOptions), membaca .env, mendaftarkan get_it Service Locator, lalu memanggil class App.
 - **app.dart**: Merupakan root dari MaterialApp. Di sinilah MultiProvider dikonfigurasi secara hierarkis.
-

Kesimpulan: Arsitektur Camelio sangat matang (*production-ready*). Pemisahan MVVM memungkinkan modul AI Gemini dapat di-maintenance atau diubah di masa depan tanpa merusak tampilan UI aplikasi, menjadikan kodenya sangat solid untuk sebuah skripsi/tugas akhir sistem cerdas.